The Journal of Open Source Software

BciPy 2.0: Experiment Orchestration, Multimodal Support, and Simulations in Python.

Tab Memmott [1,2*¶], **Matthew Lawhead** [3*], **Basak Celik** [4*], **Niklas Smedemark-Margulies** [4*], **Dylan Gaines** [5,6*], **Daniel Klee** [1*], **Carson Reader** [2*], **Allen Zhang** [2*], **Srikar Ananthoju** [4*], **and Keith Vertanen** [5*]

1 Department of Neurology, Oregon Health & Science University, Portland, OR, United States of America **2** Institute on Development and Disability, Oregon Health & Science University, Portland, OR, United States of America **3** Oregon Clinical and Translational Research Institute, Oregon Health & Science University, Portland, OR, United States of America **4** Department of Electrical and Computer Engineering, Northeastern University, Boston, MA, United States of America **5** Department of Computer Science, Michigan Technological University, Houghton, MI, United States of America **6** Department of Computer Science, Kennesaw State University, Marietta, GA, United States of America ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Sarath Menon

Submitted: 05 May 2026
Published: unpublished

Summary

Advances in Brain-Computer Interface (BCI) research require software that evolves alongside new scientific discoveries and experimental needs. BciPy 2.0 is a major update and expansion of the original BciPy 1.0, developed in response to recent progress in the field. This release addresses growing demands for multimodal integration, offline simulation, and reproducible experimental protocols, with a particular focus on communication BCIs (cBCIs). BciPy 2.0 offers robust support for multimodal data acquisition and fusion, advanced simulation capabilities, flexible task orchestration, and standardized data sharing—all within the Python ecosystem. The system prioritizes modularity and extensibility, incorporating features informed by current research trends. This manuscript provides a comprehensive overview of BciPy 2.0, including system architecture and practical usage examples.

Statement of Need

Software is the foundation of BCI research, serving as the bridge between biosignals and real-time applications that enable computer-mediated control. Reliable and adaptable tools are crucial for improving system accuracy, reducing latency, and expanding functionality—especially for communication BCI (cBCI) applications. These improvements bring cBCIs closer to practical, real-world use, with significant implications for healthcare, accessibility, and human-computer interaction.

Current trends in BCI research include increased interest in multimodal signal acquisition and integration, as well as the use of advanced modeling techniques to improve classification and inference. The scientific community is also prioritizing data practices that follow the FAIR principles — Findable, Accessible, Interoperable, and Reusable (Wilkinson et al., 2016) — which BciPy 2.0 is built to support. Additionally, some dependencies and Python versions used in BciPy 1.0 are now deprecated or incompatible with modern tools, motivating the architectural changes in BciPy 2.0. Future releases will continue to enhance interoperability with popular scientific libraries and features, and provide expanded support for experiment management.

The Journal of Open Source Software

State of the Field

A large portion of the noninvasive BCI field relies on custom-built software or established frameworks such as BCI2000 (Schalk et al., 2004) and OpenViBE (Renard et al., 2010), which provide broad support for applications like cursor control and virtual reality (VR) integration but are written in C++ and oriented toward general BCI paradigms rather than communication-specific workflows. More recent Python packages such as PyBCI (Booth et al., 2023), MetaBCI (Mei et al., 2024), and PyNoetic (Singh et al., 2025) offer EEG signal acquisition, artifact handling, and interface control, but lack integrated support for multimodal evidence fusion, language modeling, and offline simulation. BciPy occupies a distinct niche by focusing specifically on text input for communication BCIs, combining these capabilities within a single Python framework. Python's dominance in scientific computing and machine learning allows BciPy to leverage a broad ecosystem of libraries and tools, making it accessible to researchers who may not have extensive programming experience.

Software Design

BciPy supports installation on the latest versions of macOS, Linux, and Windows, with step-by-step instructions provided in the documentation and reproducible builds verified through continuous integration with GitHub Actions. Each submodule includes its own `README.md`, runnable demos, and unit tests to help users get started. Users can interact with BciPy through the client interface, by importing the package in Python, or via the PyQt6-based GUI (`BCInterface.py`, see Figure 1). The choice of interface depends on the user's coding experience and the level of customization required for their experiments.

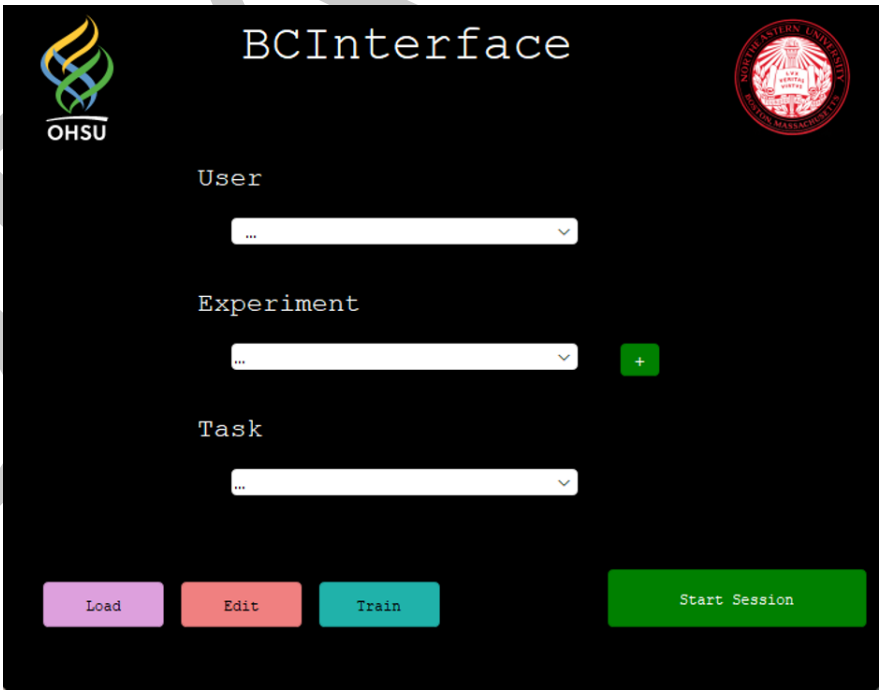

Figure 1: BciPy GUI. The BciPy GUI can be used for editing or loading parameters, training a `SignalModel`, defining a new experiment (this provides another GUI), or running an experiment or `Task`.

Experiment parameters are defined in JSON format, with default templates available in `bcipy/parameters/`. These parameters can be edited directly in the JSON files or through the graphical parameter editor shown in Figure 2, allowing researchers to configure experiment

conditions upfront and reduce input errors. Data collected with BciPy can be exported to multiple formats—including BDF, EDF, BrainVision, Brain Imaging Data Structure (BIDS), and MNE—for external analysis or sharing, with compression options to facilitate storage. These export capabilities support FAIR data principles and interoperability with common analysis tools.

BciPy leverages several scientific libraries to provide its core features, including PsychoPy, PyLSL, scikit-learn, transformers, NumPy, SciPy, Pandas, and MNE (Gramfort et al., 2014; Kothe et al., 2025; McKinney & others, 2011; Peirce, 2007; Van Der Walt et al., 2011; Virtanen et al., 2020; Wolf et al., 2020). The full list of dependencies is maintained in the `pyproject.toml` file.

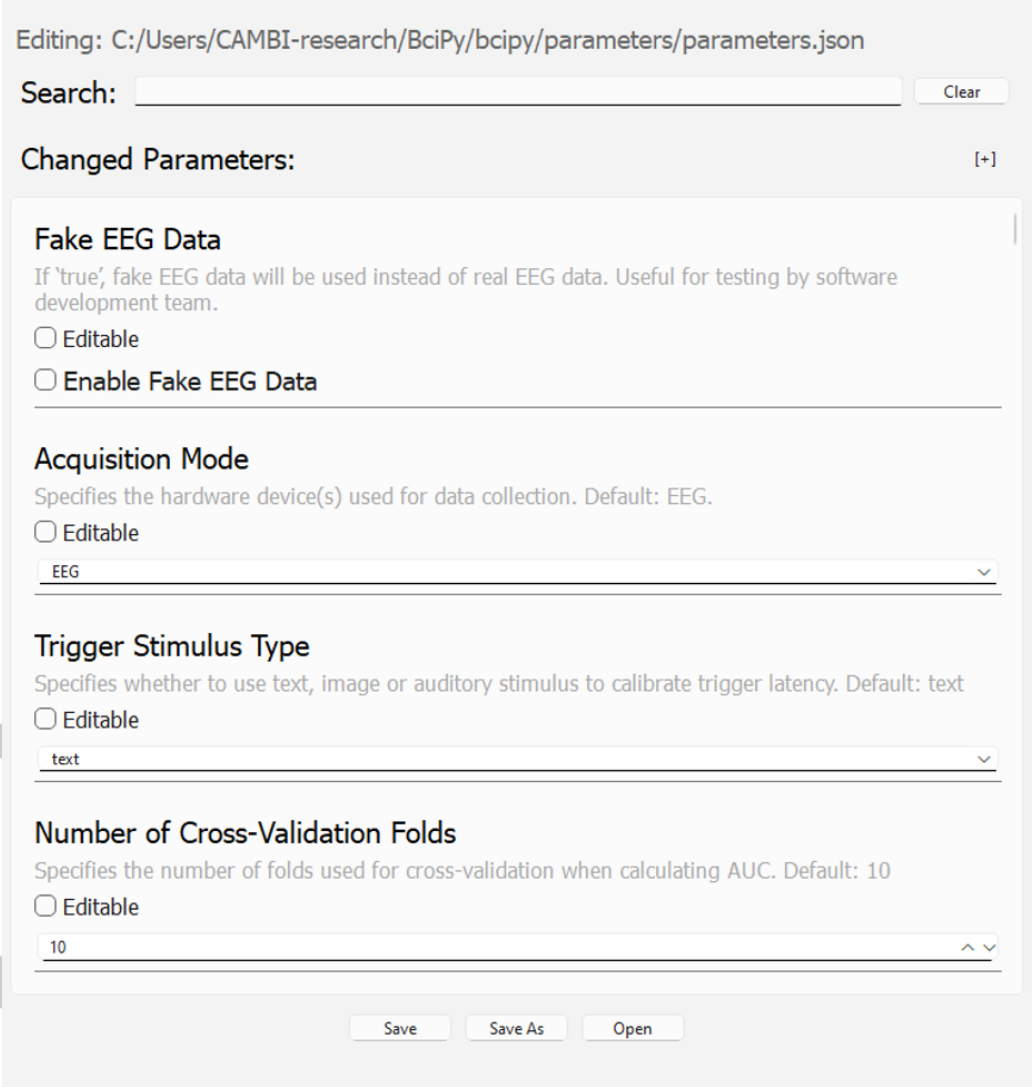

Figure 2: BciPy Parameter Editor. The BciPy Parameter Editing GUI can be used for editing, saving, or searching a parameters file. This can help prevent input errors and facilitate defining parameters for experiment conditions upfront. If parameters are changed, a panel under Changed Parameters (shown above) will display with the parameter changed and what value it's been updated to. This can help prevent accidental changes or debug issues with a set of parameters.

The Journal of Open Source Software

Research Impact Statement

The BciPy repository has made BCI research more accessible through a modular, extensible, real-time Python interface designed for practical use and reproducible experimentation. The software and accompanying documentation have been released publicly, allowing researchers to directly run the system and adapt the interface for their own BCI studies by adding new paradigms and processing methods. The repository includes example pipelines, standardized data handling utilities, and integration with common Python scientific libraries, which has helped lower the technical barrier for working with neural signals in real time.

Evidence of use is reflected in 76,000 estimated downloads (from PyPI) and 39 external forks that adapt the interface for related BCI experiments and prototyping workflows. The BciPy Python library has also been used internally by 18 developers and by collaborators to build and test closed-loop BCI applications, demonstrating that the interface is stable enough for real experimental setups rather than only proof-of-concept demonstrations. Since its initial public release (Memmott et al., 2021), BciPy has gained wide adoption in the BCI community, with 145 GitHub stars, approximately 30 citations across peer-reviewed and preprint venues. There are also more than 10 peer-reviewed publications that have used BciPy in control and clinical studies (a full list is maintained in the repository `README.md`). Early adoption of BciPy 2.0 is evident, with five published studies utilizing its redesigned architecture as of March 2026. The Python BCI ecosystem has also grown, with several complementary libraries released or updated (Booth et al., 2023; Mei et al., 2024; Singh et al., 2025; Zhu et al., 2024).

BciPy is positioned for near-term impact within the BCI community due to its emphasis on reproducibility, clear documentation, and compatibility with common hardware, software, and analysis tools. By providing a simple and extensible interface for neural data acquisition and control, our work helps accelerate rapid prototyping and experimental iteration in BCI research.

BciPy 2.0 Change Overview

BciPy 2.0 is a major update to the original BciPy 1.0, with significant improvements in architecture, functionality, and usability. The following subsections describe the key additions: task and experiment orchestration, multimodal data acquisition and evidence fusion, and a new simulation module for offline evaluation of system parameters.

Task & Experiment Support

BciPy manages the execution of experimental `Tasks` using the `SessionOrchestrator` class, which ensures `Tasks` are run in the correct order and that all data are properly persisted. Researchers can run individual `Tasks`, such as `Calibration`, directly via the client, or define complete experimental protocols for reproducible studies.

Multimodal Data Acquisition and Fusion

BciPy 2.0 introduces support for multimodal data acquisition and evidence fusion, allowing the system to consider information from multiple devices when making typing decisions. After extensive testing, we leaned more heavily on LabStreamingLayer (LSL) (Kothe et al., 2025) to support multimodal acquisition in BciPy 2.0. LSL is well-supported across the industry, with many devices providing compatible drivers. This decision allowed us to drastically simplify the acquisition module compared to BciPy 1.0 while increasing functionality.

In addition to the EEG signal model developed in BciPy 1.0, BciPy 2.0 introduces a gaze model for classification of eye gaze trajectory data that can be acquired in parallel with the EEG data stream through an eye tracker. Both the EEG and gaze models inherit the same `SignalModel` structure and are compatible with the scikit-learn estimators API (Pedregosa et al., 2011).

BciPy 2.0 expands upon the language modeling capabilities of BciPy 1.0 and removes the language model (LM) module from the previous Docker image, opting instead for direct function calls in Python. The LM module takes the text that the user has written so far and produces a likelihood distribution over the system's symbol set. This distribution can be used to present more likely characters to the user sooner, in a paradigm like RSVP Keyboard (Orhan et al., 2012), or simply to fuse with the evidence gathered from the user (e.g., EEG, gaze, etc.). BciPy 2.0 introduces inference by large language models (LLMs) via the TextSlinger API, which allows the use of causal transformer models from Hugging Face using the search algorithm from Gaines & Vertanen (2025). BciPy 2.0 also supports TextSlinger's `NGramLanguageModel` class, which leverages the KenLM package (Heafield, 2011).

BciPy Simulator

A major advancement in BciPy 2.0 is the introduction of the simulator module. This module allows researchers to use previously recorded typing data to systematically evaluate how changes in parameters, signal models, and language models impact typing performance. The simulator supports tasks such as customizing experiment parameters, conducting large-scale comparisons of language model implementations, and testing multimodal evidence fusion strategies. Additionally, the simulator framework can be extended to train new EEG signal models.

The simulator collects metrics for each run and summarizes across all runs to assess performance. It provides both a graphical user interface for designing simulation parameters and input sources as well as a command line interface for scripting and automation.

AI usage disclosure

The manuscript was written by the authors with the assistance of AI tools, including ChatGPT, Claude, Grammarly, and GitHub Copilot. The AI tools were used to help edit text and to assist with code review, documentation, testing, and formatting. The majority of BciPy core functionality was developed by the authors exclusively. The authors reviewed and edited all content generated by the AI tools to ensure accuracy and coherence.

Acknowledgements

We'd like to thank those who helped throughout the refactor, including Aida Fakhry, Ian Jackson, Julia Gangemi, Tales Imbiriba, Emma Sombers, Georgios Stratis, David Smith, Shijia Liu, Barry Oken, Deniz Erdogmus, Betts Peters, and Melanie Fried-Oken. In addition, we thank Steven Bedrick for his architectural and general advice on this significant update. This work was supported by NIH R01DC009834 and NSF IIS-1750193. Authors report no conflicts of interest.

References

Booth, L., Asghar, A., & Bateson, A. (2023). PyBCI: A python package for brain-computer interface (BCI) design. *Journal of Open Source Software*, *8*(92), 5706. https://doi.org/10.21105/joss.05706

Gaines, D., & Vertanen, K. (2025). Adapting large language models for character-based augmentative and alternative communication. *Findings of the Association for Computational Linguistics: EMNLP 2025*, 15273–15291. https://doi.org/10.18653/v1/2025.findings-emnlp.826

Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Parkkonen, L., & Hämäläinen, M. S. (2014). MNE software for processing MEG and EEG data. *Neuroimage*, *86*, 446–460. https://doi.org/10.1016/j.neuroimage.2013.10.027

Heafield, K. (2011). KenLM: Faster and smaller language model queries. *Proceedings of the Sixth Workshop on Statistical Machine Translation*, 187–197.

Kothe, C., Shirazi, S. Y., Stenner, T., Medine, D., Boulay, C., Grivich, M. I., Artoni, F., Mullen, T., Delorme, A., & Makeig, S. (2025). The lab streaming layer for synchronized multimodal recording. *Imaging Neuroscience*. https://doi.org/10.1162/IMAG.a.136

McKinney, W., & others. (2011). Pandas: A foundational python library for data analysis and statistics. *Python for High Performance and Scientific Computing*, *14*(9), 1–9.

Mei, J., Luo, R., Xu, L., Zhao, W., Wen, S., Wang, K., Xiao, X., Meng, J., Huang, Y., Tang, J., Cheng, L., Xu, M., & Ming, D. (2024). MetaBCI: An open-source platform for brain-computer interfaces. *Computers in Biology and Medicine*, *168*. https://doi.org/10.1016/j.compbiomed.2023.107806

Memmott, T., Koçanaoğulları, A., Lawhead, M., Klee, D., Dudy, S., Fried-Oken, M., & Oken, B. (2021). BciPy: Brain–computer interface software in python. *Brain-Computer Interfaces*, *8*(4), 137–153. https://doi.org/10.1080/2326263x.2021.1878727

Orhan, U., Hild, K. E., Erdogmus, D., Roark, B., Oken, B., & Fried-Oken, M. (2012). RSVP keyboard: An EEG based typing interface. *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 645–648. https://doi.org/10.1109/ICASSP.2012.6287966

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Peirce, J. W. (2007). PsychoPy—psychophysics software in python. *Journal of Neuroscience Methods*, *162*(1-2), 8–13. https://doi.org/10.1016/j.jneumeth.2006.11.017

Renard, Y., Lotte, F., Gibert, G., Congedo, M., Maby, E., Delannoy, V., Bertrand, O., & Lécuyer, A. (2010). OpenViBE: An open-source software platform to design, test, and use brain-computer interfaces in real and virtual environments. *Presence*, *19*(1), 35–53. https://doi.org/10.1162/pres.19.1.35

Schalk, G., McFarland, D. J., Hinterberger, T., Birbaumer, N., & Wolpaw, J. R. (2004). BCI2000: A general-purpose brain-computer interface (BCI) system. *IEEE Transactions on Biomedical Engineering*, *51*(6), 1034–1043. https://doi.org/10.1109/TBME.2004.827072

Singh, G., Chharia, A., Upadhyay, R., Kumar, V., & Longo, L. (2025). PyNoetic: A modular python framework for no-code development of EEG brain-computer interfaces. *PLOS ONE*, *20*(8), e0327791. https://doi.org/10.1371/journal.pone.0327791

Van Der Walt, S., Colbert, S. C., & Varoquaux, G. (2011). The NumPy array: A structure for efficient numerical computation. *Computing in Science & Engineering*, *13*(2), 22–30. https://doi.org/10.1109/mcse.2011.37

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Wilkinson, M. D., Dumontier, M., Aalbersberg, Ij. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, E., Silva Santos, L. B. da, Bourne, P. E., & others. (2016). The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, *3*(1), 160018. https://doi.org/10.1038/sdata.2016.18

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., & others. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45. https://doi.org/10.18653/v1/2020.emnlp-demos.6

Zhu, H., Beierholm, U., & Shams, L. (2024). BCI toolbox: An open-source python package for the bayesian causal inference model. *PLOS Computational Biology*, *20*(7), e1011791. https://doi.org/10.1371/journal.pcbi.1011791